Laboratório Prático: Prevendo o Preço de um Imóvel

Regressão Múltipla, Variáveis Dummy, Limpeza e Seaborn

Prof. João Vítor Rodrigues de Vasconcelos

Engenharia de Computação / Análise e Desenvolvimento de Sistemas
joaovasconcelos@univicosa.com.br

ADS308

Objetivos do Laboratório (Atualizado)

Subindo o Nível

Nesta prática, usaremos uma base de dados maior (300 registros) e mais complexa para simular um desafio real de Ciência de Dados.

- Consolidar o uso da **Regressão Linear Múltipla**.
- Realizar **Análise Exploratória de Dados (EDA)** com a biblioteca **Seaborn**.
- Lidar com **Variáveis Categóricas** (textos) aplicando a técnica de *One-Hot Encoding* (Variáveis Dummy).
- Limpar dados inconsistentes e ruidosos (Data Wrangling).
- Calcular e interpretar o **RMSE** no contexto financeiro.

O Cenário Prático

- A imobiliária de Viçosa-MG expandiu sua base de dados.
- Agora temos acesso a 1000 registros históricos de vendas.
- Além da área e número de quartos, o banco de dados agora inclui o **Tipo do Imóvel** (Apartamento, Cobertura, Casa, etc.).
- O objetivo permanece: criar um modelo de Machine Learning para prever o **Preço (em milhares de Reais)**.

As Variáveis do Modelo

Features (Variáveis Preditoras)

1. `Area_m2`: Área total do imóvel (Numérica Contínua).
2. `Num_Quartos`: Quantidade de quartos (Numérica Discreta).
3. `Tipo_Imovel`: Categoria do imóvel (Variável Texto - Novo Desafio!).
4. `Idade_Imovel`: Tempo desde a construção (Variável Suja - Requer Limpeza!).

Target (Variável Alvo)

`Preco_Mil`: Valor do imóvel em milhares de R$ (ex: 250 = R$ 250.000,00).

Obtendo os Dados

Download do Dataset

O arquivo `dataset_imoveis_vicosa.csv` já foi gerado e está disponível para download. Façam o download e coloquem na mesma pasta do Jupyter Notebook.

Link para download da base de dados:
Link do Google Drive

Carregando no Pandas

```
import pandas as pd

# Carregando o dataset
df = pd.read_csv("dataset_imoveis_vicosa.csv")
print("Dados carregados com sucesso! Tamanho:", df.shape)
```

Passo 1: Análise Exploratória (Pandas)

- O primeiro passo é sempre inspecionar o que o Pandas interpretou dos seus dados.

```
# Verificando os tipos de dados e valores nulos
print(df.info())

# Inspecionando as primeiras linhas
display(df.head(10))
```

Atenção aos Problemas:

Observem que a coluna `Idade_Imovel` está como *object* e possui valores como "NaN", "novo", "anos"e até números negativos.

Passo 2: Visualização com Seaborn (Target)

- Vamos entender a distribuição dos preços na nossa cidade usando gráficos estatísticos.

```
import matplotlib.pyplot as plt
import seaborn as sns

sns.set_theme(style="whitegrid")

plt.figure(figsize=(8, 5))
sns.histplot(df["Preco_Mil"], kde=True, bins=20, color="blue")
plt.title("Distribuição dos Preços dos Imóveis")
plt.xlabel("Preço (Milhares de R$)")
plt.ylabel("Frequência")
plt.show()
```

Passo 2: Visualização com Seaborn (Boxplot)

- Como a categoria do imóvel afeta o preço final? O Boxplot revela isso perfeitamente.

```
plt.figure(figsize=(10, 6))
sns.boxplot(x="Tipo_Imovel", y="Preco_Mil", data=df)
plt.title("Preço do Imóvel por Tipo (Categoria)")
plt.xticks(rotation=15)
plt.show()
```

Análise Visual

Anotem no Notebook: Qual tipo de imóvel possui a maior mediana de preço?

Passo 2: Visualização com Seaborn (Scatter)

- Vamos buscar a correlação linear que o nosso modelo tentará aprender.

```
plt.figure(figsize=(10, 6))
sns.scatterplot(x="Area_m2", y="Preco_Mil", hue="Tipo_Imovel",
                size="Num_Quartos", sizes=(20, 150),
                data=df, alpha=0.8)
plt.title("Relação entre Área, Tipo e Preço")
plt.legend(bbox_to_anchor=(1.05, 1), loc="upper left")
plt.tight_layout()
plt.show()
```

Passo 3: Limpeza da Variável Suja

O Desafio da `Idade_Imovel`

Modelos matemáticos falham se receberem textos ou nulos. Vocês precisam criar uma rotina de limpeza.

- **Remover Strings:** Extraiam a palavra "anos"(usem `.str.replace()`).
- **Padronizar Textos:** Substituam "novo"por 0. Transformem "Desconhecida"em `NaN`.
- **Tratar Valores Absurdos:** Convertam a coluna para numérico e removam/corrijam idades negativas.
- **Lidar com Nulos (NaN):** Imputem os valores ausentes com a **mediana** da coluna.

O Problema das Variáveis Categóricas

- O nosso dataset agora tem a coluna `Tipo_Imovel` (ex: "Casa Padrão", "Apartamento Cobertura").
- A equação da Regressão Linear Múltipla é:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 ...$$

- **Pergunta:** Como multiplicamos um peso (β) pela palavra "Casa"?
- **Resposta:** Não multiplicamos. Precisamos converter textos em números binários.

Solução: One-Hot Encoding (Variáveis Dummy)

O que é One-Hot Encoding?

A técnica cria uma nova coluna para cada categoria possível, preenchendo com 1 (se verdadeiro) ou 0 (se falso).

No Pandas, fazemos isso em uma única linha após limpar os dados:

```
# Convertendo a coluna qualitativa em colunas quantitativas
df = pd.get_dummies(df, columns=["Tipo_Imovel"], drop_first=True)

# Vejam o resultado
display(df.head())
```

Nota: O parâmetro `drop_first=True` evita a armadilha da multicolinearidade (Dummy Variable Trap), removendo uma das categorias que servirá como base.

Passo 4: Matriz de Correlação (Pós-Limpeza)

DESAFIO

É necessário arrumar a coluna idade para continuar a partir daqui

- Apenas **depois** de limpar a idade e aplicar o `get_dummies`, podemos calcular as correlações de tudo.

```
plt.figure(figsize=(10, 8))
# Calculando a correlação de Pearson
matriz_corr = df.corr()

# Plotando o Heatmap
sns.heatmap(matriz_corr, annot=True, cmap="coolwarm", fmt=".2f")
plt.title("Matriz de Correlação")
plt.show()
```

Passo 5: Divisão em Treino e Teste

- Chegou a hora do Machine Learning. Vamos isolar as features (X) e o alvo (y).

```
from sklearn.model_selection import train_test_split

# O Target é o Preco_Mil. Todo o resto é Feature.
X = df.drop("Preco_Mil", axis=1)
y = df["Preco_Mil"]

# Split de 80% treino / 20% teste
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

Passo 6: Treinamento do Modelo (OLS)

- Vamos instanciar e ajustar a Regressão Linear aos dados de treino.

```
from sklearn.linear_model import LinearRegression

# Instanciando o modelo
modelo = LinearRegression()

# Treinando (Calculando os Mínimos Quadrados)
modelo.fit(X_train, y_train)

# Inspecionando
print("Intercepto:", modelo.intercept_)
print("Coeficientes:", modelo.coef_)
```

Passo 7: Métricas de Erro (RMSE)

Root Mean Squared Error

Fazemos as previsões na base de Teste e calculamos a raiz quadrada do Erro Quadrático Médio.

```
import numpy as np
from sklearn.metrics import mean_squared_error

# Predições
y_pred = modelo.predict(X_test)

# Calculando o MSE e RMSE
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)

print(f"MSE: {mse:.2f}")
print(f"RMSE: {rmse:.2f}")
```

Passo 8: O Teste de Fogo (R^2 e Previsão)

O Coeficiente de Determinação (R^2)

Além do erro em Reais (RMSE), calculem o R^2 para saber qual a porcentagem da variação dos preços é explicada pelas nossas features.

```
from sklearn.metrics import r2_score

# Calculando o R^2 no conjunto de teste
r2 = r2_score(y_test, y_pred)
print(f"R^2 Score: {r2:.4f} (ou {r2*100:.2f}%)")
```

A Interpretação de Negócio

O que vocês devem escrever no Notebook:

Transformem a matemática em valor para a empresa.

- Lembrem-se que nossa variável alvo está em **milhares de Reais**.
- **Interpretando o Erro (RMSE):** Se o seu `RMSE` deu 35.5, isso significa que: *"Em média, as previsões do modelo erram o preço do imóvel por aproximadamente R$ 35.500,00."*
- **Interpretando o Ajuste (R^2):** Se o seu R^2 deu 0.82, isso significa que: *"82% da variação nos preços dos imóveis é explicada pelas características que fornecemos (área, quartos, idade e tipo)."*
- **A tarefa final:** Analisem os coeficientes gerados, o RMSE e o R^2. O modelo já pode ser colocado em produção na imobiliária? Justifiquem sua decisão técnica e de negócio.

Desafio Prático

Desafio Prático: Colocando o Modelo à Prova!

Utilizem o método `modelo.predict()` para responder à seguinte pergunta de negócio:

"Qual é o preço estimado pelo seu modelo para um Apartamento Padrão, com 75m² de área, 2 quartos e 5 anos de construção?"

Dica de Ouro: Lembrem-se de que o modelo treinado agora possui as colunas Dummy. Vocês precisarão passar um *array* com as características na **mesma ordem** das colunas do `X_train` (preenchendo com 0 as categorias que não se aplicam).

Desafio Prático: Prevendo Novos Valores (Na Prática)

A "Pegadinha"das Variáveis Dummy

O Scikit-Learn exige que os novos dados tenham **exatamente o mesmo número e a mesma ordem** das colunas que o modelo viu durante o treinamento (`X_train`).

- Primeiro, imprimimos as colunas para descobrir a assinatura exata do modelo:

```
# Verificando a ordem gerada pelo Pandas
print(X_train.columns)

# Saída esperada:
# ["Area_m2", "Num_Quartos", "Idade_Imovel",
#  "Tipo_Imovel_Apartamento Padrão", "Tipo_Imovel_Casa Padrão",
#  "Tipo_Imovel_Casa com Quintal"]
```

Desafio Prático: Prevendo Novos Valores (Na Prática)

- Agora, criamos uma matriz 2D (lista dentro de lista) respeitando essa ordem para o nosso Apartamento Padrão de 75m², 2 quartos e 5 anos:

```
# Mapeamento: Área=75, Quartos=2, Idade=5, Apto=1, CasaP=0, CasaQ=0
caracteristicas_cliente = [[75, 2, 5, 1, 0, 0]]

preco_previsto = modelo.predict(caracteristicas_cliente)
print(f"Preço estimado: R$ {preco_previsto[0]:.2f} mil")
```

Bom trabalho a todos e mãos no código!

Dúvidas? joaovasconcelos@univicosa.com.br